

CIS5300 Term Project: Toxic Comment Detection

Serena Gu, Tingnan Hu, Ruohan Zhang, Liulu Yue

Motivation

Online platforms have become essential spaces for sharing ideas, debating opinions, and building communities. However, the rise of abusive and toxic language has made these interactions increasingly unsafe and discouraging for many users. Toxic comments, which are rude, disrespectful, or likely to make someone withdraw from a discussion, pose a major challenge to maintaining healthy online discourse. As a result, many communities struggle to moderate content effectively, sometimes resorting to disabling comment sections altogether. Our project aims to address this issue by developing a model for automatic toxic comment detection, contributing to safer and more inclusive online environments.

Problem Definition and Goals

The goal of this project is to build a binary classifier that determines whether a user-generated comment is toxic or non-toxic. Formally, given an input string x , we aim to learn a function $f(x) \rightarrow \{0,1\}$, where 1 indicates toxic content. This task is challenging from an NLP perspective because toxicity can be expressed implicitly, through sarcasm, slang, or subtle phrasing. Building an accurate and fair model therefore requires careful modeling of language, context, and bias. The output of this system could help automate content moderation and reduce the manual burden on human moderators.

Dataset

We will use the publicly available dataset from the Kaggle Toxic Comment Classification Challenge, hosted by Google and Jigsaw. The [dataset](#) contains a large number of Wikipedia comments that have been labeled by human raters for toxic behavior. We will primarily use the binary label (toxic vs. non-toxic) for this project. The dataset is well-suited for our task because it covers a range of real-world conversational scenarios and contains examples of both explicit and subtle toxicity.

Planned Approach

Data Preprocessing:

- Clean and normalize text (lowercasing, removing special characters, handling URLs/emojis).
- Handle contractions and lemmatization to unify different word forms and improve token consistency. Remove stop words where appropriate, while testing whether keeping them improves contextual understanding for transformer-based models.
- Tokenize using both traditional NLP tokenizers (e.g., spaCy, NLTK) and subword-based tokenizers (e.g., WordPiece or Byte Pair Encoding) for compatibility with modern embeddings.
- Split data into training, validation, and test sets with stratified sampling to preserve class distribution.

Baseline Model: We will begin with a simple baseline classifier, such as majority class baseline (always predict non-toxic), Logistic Regression, or Naive Bayes with Bag-of-Words or TF-IDF features. This will establish a performance reference point.

Reproduction of a Strong Baseline: Next, we will reproduce a stronger model from existing literature or competition solutions. Potential approaches include:

- LSTM or BiLSTM models using pre-trained GloVe or FastText embeddings to capture sequential and semantic information.
- Fine-tuning a transformer-based model such as BERT or DistilBERT, which have shown state-of-the-art performance in text classification tasks. This involves adding a classification head on top of the pretrained model and fine-tuning on the Kaggle dataset.
- We will experiment with hyperparameter tuning (learning rate, batch size, dropout rate, and max sequence length) to optimize model performance.

This step ensures we meet the requirement to replicate a known strong baseline before introducing innovations.

Illustrative Example

Example 1

- **Input:** “Bye! Don't look, come or think of coming back! Tosser.”
- **Output:** toxic

Example 2:

- **Input:** “Hey man, I'm really not trying to edit war. It's just that this guy is constantly removing relevant information and talking to me through edits instead of my talk page. He seems to care more about the formatting than the actual info.”
- **Output:** not toxic